

Assignment No : 04 - Linear Regression for Home Price Prediction

Problem Statement:

Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset (<https://www.kaggle.com/c/boston-housing>). The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this dataset. The objective is to predict the value of prices of the house using the given features

Step 1 : Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Step 2 : Upload and Load Dataset

```
df = pd.read_csv('/content/HousingData.csv')
df.head()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1	296	15.3	396.90
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2	242	17.8	396.90
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3	222	18.7	396.90

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

Step 3: Check Dataset Information

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   CRIM        486 non-null   float64
 1   ZN          486 non-null   float64
 2   INDUS       486 non-null   float64
 3   CHAS        486 non-null   float64
 4   NOX         506 non-null   float64
 5   RM          506 non-null   float64
 6   AGE         486 non-null   float64
 7   DIS         506 non-null   float64
 8   RAD         506 non-null   int64   
 9   TAX         506 non-null   int64   
10  PTRATIO     506 non-null   float64
11  B           506 non-null   float64
12  LSTAT       486 non-null   float64
13  MEDV        506 non-null   float64
dtypes: float64(12), int64(2)
memory usage: 55.5 KB
```

```
df.size
```

```
7084
```

```
df.shape
```

```
(506, 14)
```

```
df.columns
```

```
Index(['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE', 'DIS', 'RAD', 'TAX',
       'PTRATIO', 'B', 'LSTAT', 'MEDV'],
      dtype='object')
```

Step 4: Summary Statistics

```
df.describe()
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	
count	486.000000	486.000000	486.000000	486.000000	506.000000	506.000000	486.
mean	3.611874	11.211934	11.083992	0.069959	0.554695	6.284634	68.
std	8.720192	23.388876	6.835896	0.255340	0.115878	0.702617	27.
min	0.006320	0.000000	0.460000	0.000000	0.385000	3.561000	2.
25%	0.081900	0.000000	5.190000	0.000000	0.449000	5.885500	45.
50%	0.253715	0.000000	9.690000	0.000000	0.538000	6.208500	76.
75%	3.560263	12.500000	18.100000	0.000000	0.624000	6.623500	93.
max	88.976200	100.000000	27.740000	1.000000	0.871000	8.780000	100.

Step 5: Handle Missing Values

```
df = df.fillna(df.mean())
```

Step 6: Separate Features and Target

```
X = df.drop("MEDV", axis=1)
y = df["MEDV"]
```

Step 7: Split Training and Testing Data

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Step 8: Create Linear Regression Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```

▼ LinearRegression ⓘ ?

```
LinearRegression()
```

Step 9: Predict House Prices

```
y_pred = model.predict(X_test)
```

Step 10: Display Predicted Values

```
print(y_pred[:10])
```

```
[29.14324958 36.53566752 14.49251294 25.08111038 18.45609233 23.01049373  
18.13868607 14.62651674 22.1158938 20.87063644]
```

Step 11: Compare Actual vs Predicted Prices (Optional but Good)

```
comparison = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})  
comparison.head()
```

	Actual	Predicted	
173	23.6	29.143250	
274	32.4	36.535668	
491	13.6	14.492513	
72	22.8	25.081110	
452	16.1	18.456092	

Next steps:

[Generate code with comparison](#)[New interactive sheet](#)

Step 12: Evaluate the Model

```
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))  
print("R2 Score:", r2_score(y_test, y_pred))
```

```
Mean Squared Error: 25.01767202384268  
R2 Score: 0.6588520195508143
```

Step 13: Visualization (Actual vs Predicted)

```
plt.scatter(y_test, y_pred)  
plt.xlabel("Actual Prices")  
plt.ylabel("Predicted Prices")  
plt.title("Actual vs Predicted House Prices")  
plt.show()
```



Conclusion

In this practical, a Linear Regression model was implemented to predict housing prices using the Boston Housing dataset. The dataset was preprocessed, split into training and testing sets, and the model performance was evaluated using Mean Squared Error and R^2 score.